POLITECNICO
MILANO 1863

AN2DL - First Challenge Report
Dropout Survivors

Mengshuang Tang, Paradise Pourhamzeh, Xiaohan Pan,
Mengshuang Tang, Codabench Nickname2, Elena_Pan
270498, 271235 274914

November 17, 2025

1 Introduction

This project focuses on *time series classification* and *the use of deep learning techniques*.

- Cleaning the dataset
- Implementation of a Custom **Bi-LSTM** from Scratch
- Applying ***Fine-Tuning*** techniques

2 Problem Analysis

The provided dataset consists of **661 samples**, labeled into three classes representing different levels of pain severity. Each sample comprises **a time-series of 160 timesteps**. At every time step, three distinct categories of features are recorded:

- responses to ***four*** pain-related surveys
- composition of ***three*** body segments;
- sensor data from ***31*** different joints

To build a robust deep learning model, we need to perform data analysis and cleaning:

1. The dataset is characterized by a severe class **imbalance**, as evidenced by an analysis of its distribution (as shown in Figure 1).
2. The initial dataset contains a large number of raw features. To prepare the data for modeling, a comprehensive **feature engineering** process should be performed.

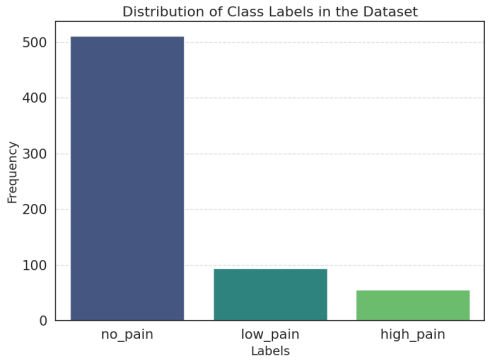

Figure 1: Dataset Distribution

To address the issue of class imbalance, two primary strategies were employed. First, **stratified sampling** was utilized for the train-validation split. Second, **class weights**[4] were calculated and incorporated into the loss function. As part of the data processing, **zero variance features** (i.e., constant columns) were removed because they do not provide discriminatory information. Furthermore, for each sample index, the corresponding **body composition features** were aggregated into a unified representation and then converted to a numerical format using the **hot encoding**.

3 Method

3.1 Feature Separation

The model processes two distinct input streams: temporal and static. The **time-dependent features** were fed into the recurrent layers to extract sequential patterns. In parallel, the **time-invariant body part information** was concatenated with the final hidden state of the recurrent layers.

3.2 Model Architecture

We split the dataset into a training set with 80% and a validation set with 20%. To avoid overfitting, An **Early Stopping protocol** was employed.

3.2.1 LSTM

A model employing a standard, unidirectional Long Short-Term Memory (LSTM) network was first developed, as our performance benchmark with a foundational recurrent architecture. However, when evaluated on the validation set, this model exhibited limited performance.

3.2.2 Bi-LSTM

A Bidirectional LSTM (Bi-LSTM) was implemented to achieve two simultaneous goals: enhancing the model's **contextual understanding** and improving its generalization capabilities. We developed the model with 2 blocks: - a **forward** LSTM and a **backward** LSTM with 1 hidden layer - a **Softmax** layer for the final prediction we processed the input sequence from the beginning to the end and in reverse according to Bi-LSTM. For Softmax, This function maps the unbounded logits to a set of normalized probability scores, one for each class. Mathematically, the softmax function computes the probability for the i-th class, p_i, from the logit vector z as follows:

$$p_i = \text{softmax}(z)_i = \frac{\exp(z_i)}{\sum_{j=1}^{K} \exp(z_j)} \tag{1}$$

3.3 Regularization Strategies

To mitigate overfitting, a comprehensive Dropout[3] strategy was employed, targeting different components of the network: - input Dropout: A dropout mask was applied to the connections between the network's input and the Bi-LSTM layer. - Hidden Dropout: A variational dropout mask was applied to the hidden-to-hidden connections within the Bi-LSTM layer itself. - Standard Dropout: a conventional dropout layer was placed between the output of the Bi-LSTM layer and the final classification layer. To further regularize the model, the loss function was augmented with an **L2 regularization term** (or weight decay) and $\lambda = 0.001$. The magnitude of this weight decay penalty was controlled by a lambda coefficient. This method penalizes large weights and improves the generalization performance.

3.4 Model with Fine-Tuning

Starting from the Bi-LSTM model, upon initial implementation, it's performance was benchmarked on the validation set, achieving an **F1 score** of **0.89**. This score reflects the model's baseline capability to balance precision and recall on unseen data, providing a starting point for further optimization.
A heuristic tuning strategy was adopted for hyperparameter optimization. The fine-tuning process was guided by observing the model's behavior during training, primarily through the visualization of learning curves (loss and F1 score). Adjustments to key hyperparameters were made iteratively based on the insights derived from these plots, such as identifying signs of overfitting or suboptimal convergence speed.
For the construction of the validation and test sets, **non-overlapping windows** were extracted by setting the stride equal to the window size. This was done to ensure sample independence and prevent an overestimation of the model's performance.
The final, optimized model achieved a classification accuracy of **94%** on the held-out test set.

4 Experiment

4.1 Other attempts

During our experiments, We also experimented with models other than LSTMs to better compare how different architectures handle this dataset as follow: In order to improve feature extracting ability, we implemented a CNN[2] and based on that, we further explored different combinations of Transform-

Table 1: Final models. Best results are highlighted in **bold**.

Model	Validation Accuracy(%)	Test Accuracy(%)
Baseline CNN (1D conv)	88.49	87.10
LSTM	88.50	86.95
CNN + BiLSTM (no attention)	79.86	80.03
CNN + BiLSTM + Attention	91.15	92.08
CNN + BiLSTM + Transformer + Attention	90.79	90.73
CNN + BiLSTM + Transformer + Attention + Residual + Augmentation	92.04	90.99
Bi-LSTM	**94.01**	**94.17**

ers, and Bi-LSTMs, along with attention mechanisms, residual connections, and data augmentation.

4.2 Failed Experiment

we also implemented other strategies and techniques. However, neither resulted in any significant improvement:

- **Outliers Handling:** There are some sensor data recorded as zeroes in the dataset, which we hypothesized to be caused by transient sensor disconnections. We attempted adjust these anomalies through interpolation.

- **Embedding of Categorical Features[1]:** For features such as pain_survey, whose numerical values do not necessarily reflect linear relationships, we explored the use of embedding layers to reduce the influence of arbitrary discrete scales.

- **Pearson Correlation:** Analysis identified a strong correlation ($\rho = 0.95$) between joint_10 and joint_11. However, according to an ablation study, in which one of these columns was removed, resulted in a decrease in model performance.

- **Feature Construction:** We tested on the four pain survey columns, where their mean and standard deviation were used as composite features to represent the overall pain state.

5 Results & Discussion

After testing the model, our model evolved substantially to reached nearly **95%** test accuracy. And the combinations of different models mentioned before all produced competitive results as well. In table 1 there are some scores of the final model.
At the early stage, the model fit the training data well but exhibited poor generalization to the test set. This performance gap was largely driven by misclassification of minority classes, whose limited representation caused the model to bias toward the majority labels.
We also recognized that not every combination of model components or iterative application of various mechanisms leads to performance gains. Each task requires its own targeted analysis, and it is often preferable to use a more streamlined model with fewer parameters whenever possible.

6 Conclusions

This work highlights the crucial role of data preprocessing in building a robust system.
Although the results are promising, several directions remain open for further exploration:

- Developing improved resampling strategies for samples with extremely limited data.

7 Contributions

All team members jointly contributed to the code development and the writing of the report.

References

[1] F. Alharbi, L. Ouarbya, and J. A. Ward. Comparing sampling strategies for tackling imbalanced data in human activity recognition. *Sensors*, 22(4), 2022.

[2] V. Raj, S. Magg, and S. Wermter. Towards effective classification of imbalanced data with convolutional neural networks. In F. Schwenker, H. M. Abbas, N. El Gayar, and E. Trentin, editors, *Artificial Neural Networks in Pattern Recognition*, pages 150–162, Cham, 2016. Springer International Publishing.

[3] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov. Dropout: a simple way to prevent neural networks from overfitting. *J. Mach. Learn. Res.*, 15(1):1929–1958, Jan. 2014.

[4] j. sylvaincom. Stratifiedshufflesplit. `https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.StratifiedShuffleSplit.html`. Accessed: 2025-01-01.